FATEC POMPEIA

Introdução à Computação em Nuvem · ISW055 · 161_SIST. INTELIGENTES_N

P2

Relatório consolidado

arquitetura e atividades do 2º bimestre

Avaliação individual · 2026.2

Nicollas Zoratto de Carvalho

Produto: **StudyMatch**
https://github.com/NicollasZoratto/Smart-STUDY-Match

Prof. Allan Lincoln Rodrigues Siriani
Pompeia, 12/11/2026

Sumário

1 Introdução 2
2 Metodologia 2
3 Arquitetura do SaaS 2
3.1 Visão geral e componentes 2
3.2 Onde roda 3
3.3 Fluxo de uma requisição 3
3.4 Fluxo de assinatura 4
3.5 Modelo de dados do domínio 4
3.6 Segurança 5
3.7 Observabilidade, CI/CD e documentação da API 5
3.8 Camadas IaaS, PaaS e SaaS do projeto 5
4 Quadro de entregas 6
5 Atividades realizadas 6
5.1 Atividade 7 — Plano Premium — cobrança de verdade com Stripe 6
5.2 Atividade 8 — SaaS, PaaS, IaaS — e a documentação que o projeto nunca teve 7
5.3 Atividade 9 — Fork e plano de negócio 9
5.4 Atividade 10 — Seu negócio no ar — o domínio no lugar dos filmes 10
5.5 Atividade 11 — SaaS completo — planos, limites e assinatura com Stripe 12
6 Considerações finais 15
7 Declaração de autoria 15

1 Introdução

A disciplina Introdução à Computação em Nuvem (ISW055), da Fatec Pompeia, propõe que o aluno construa, ao longo do semestre, um sistema web completo e o coloque no ar usando os serviços típicos da nuvem: containers orquestrados pelo Portainer, banco de dados gerenciado, armazenamento de objetos, fila de eventos, autenticação por microsserviço e cobrança por assinatura. No primeiro bimestre o sistema foi um catálogo de filmes do Tom Hanks; neste segundo bimestre ele deixou de servir a um catálogo e passou a servir a um negócio.

O negócio escolhido foi o **StudyMatch**: uma plataforma que une estudantes em grupos de estudo por disciplina e horário, pensada para quem trabalha de dia e estuda à noite. A dor que ele resolve é a dificuldade de achar colegas da mesma matéria com horário em comum, hoje improvisada em grupos de WhatsApp que esfriam em poucas semanas. O repositório foi copiado com todo o histórico para um repositório novo e passou a ter papéis do negócio (administrador, líder de grupo e estudante), API própria, limites por plano e assinatura recorrente no Stripe, em modo de teste.

Este relatório tem duas metades. A primeira (seção 3) descreve a arquitetura do sistema **que está no ar**, não um projeto ideal: componentes, onde roda, o caminho de uma requisição, o ciclo de vida de uma assinatura, o modelo de dados, a segurança e as camadas IaaS, PaaS e SaaS. A segunda (seções 4 e 5) é o quadro das atividades 7 a 11, com data planejada, data realizada e uma ficha com a evidência de cada entrega, inclusive as que saíram com atraso.

2 Metodologia

As atividades 7 e 8 foram feitas no repositório público do catálogo de filmes (NicoFlix), o mesmo do primeiro bimestre. Na atividade 9 esse repositório foi copiado, com todos os commits, para um repositório novo com o nome do produto, onde o SaaS foi desenvolvido nas atividades 10 e 11, na mesma stack do Portainer e no mesmo domínio. Como o histórico foi copiado inteiro, os commits das atividades 7 e 8 têm o mesmo hash nos dois repositórios, e os links deste relatório apontam para o repositório novo.

As datas e horas de cada entrega vêm do histórico do Git (`git log`, no fuso de Brasília), conferidas na página do commit no GitHub e, da atividade 9 em diante, na issue de entrega que menciona o professor, cuja data de abertura também vale como evidência. As atividades 7 e 8 usam o horário do commit; as atividades 9, 10 e 11 usam o horário da issue de entrega. Os prints mostram a página do commit ou da issue com a data visível e o resultado do sistema em execução. Dados pessoais (e-mail e nome do titular nos prints do Stripe) foram borrados.

O código e a documentação foram escritos com o apoio do assistente de IA Claude (Anthropic), usado como ferramenta de programação e redação; as decisões, a configuração do Portainer, do Stripe e do GitHub e os testes no sistema no ar foram feitos por mim. O sistema tem testes automatizados com pytest (61 testes na última entrega), que rodam sem banco nem Stripe reais, e a verificação final foi manual, no domínio publicado.

3 Arquitetura do SaaS

3.1 Visão geral e componentes

O sistema é composto por três serviços Flask (app, auth-service e log-service), um Redis, um MinIO e um banco MariaDB externo, além do Stripe, do Mailtrap e do GitHub como serviços de terceiros. Só o app é público pelo domínio; o auth-service, o log-service e o Redis existem apenas na rede interna do Docker. A API do MinIO tem uma porta publicada porque o navegador baixa as imagens direto dele, por URL pré-assinada e temporária. O Stripe fala com o app por webhook assinado.

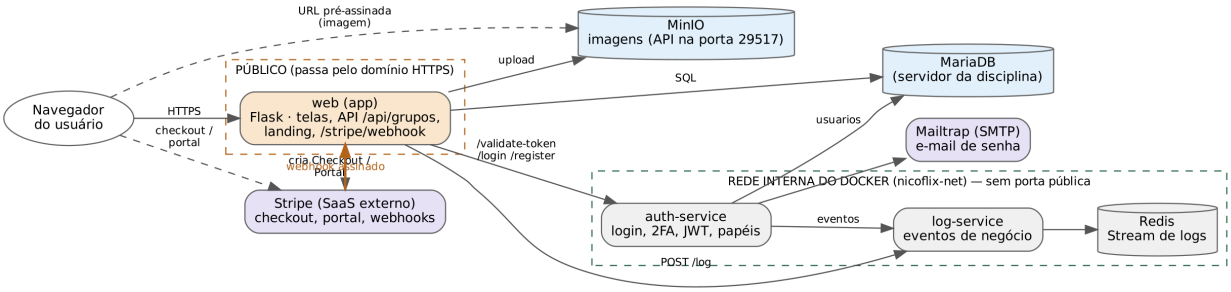


Figura 1: Diagrama de componentes e microsserviços do SaaS.

Componente	Responsabilidade	Tecnologia / imagem	Público?
app (`web`)	telas, API REST de grupos, landing page, checkout, portal e webhook do Stripe, painel admin	Flask, Python 3.10	sim (domínio)
auth-service	cadastro, login com 2FA, JWT, papéis, redefinição de senha por e-mail	Flask, JWT HS256	não (interno, :5001)
log-service	recebe e lista eventos de negócio e auditoria	Flask	não (interno, :5002)
Redis	armazenamento dos logs (Redis Stream, até 10.000 eventos)	redis:7-alpine	não (interno)
MariaDB	usuários, grupos, disciplinas, perfis, assinaturas	MariaDB externo da disciplina	não (acesso por credencial)
MinIO	fotos de perfil e capas dos grupos	bitnamilegacy/minio	só a API (:29517), via URL assinada
Stripe (externo)	checkout, assinaturas, portal, webhooks	serviço SaaS	webhook ⟶ app

3.2 Onde roda

O sistema roda em uma única stack do Portainer (stack `smartstudymatch`), na mesma VM e no mesmo domínio usados desde o começo do semestre: https://nicollas-carvalho-isw055.lapps.studio. A stack puxa o código do repositório novo (https://github.com/NicollasZoratto/Smart-STUDY-Match, branch `main`) e o Portainer faz o build das imagens a cada *Pull and redeploy*. São cinco containers (`nicoflix_web`, `nicoflix_auth`, `nicoflix_logs`, `nicoflix_redis` e `nicoflix_minio`; os nomes dos containers foram mantidos do projeto anterior) em uma rede interna `nicoflix-net`, mais o volume `minio_data` para os arquivos. O limite de memória por container é o do enunciado (1 GB), aplicado pela infraestrutura, e não configurado no `docker-compose.yml`.

As configurações sensíveis entram por variáveis de ambiente da stack. Os nomes são: `DB_HOST`, `DB_USER`, `DB_PASSWORD`, `DB_NAME`, `JWT_SECRET`, `SECRET_KEY`, `SMTP_HOST`, `SMTP_PORT`, `SMTP_USER`, `SMTP_PASS`, `STRIPE_SECRET_KEY`, `STRIPE_WEBHOOK_SECRET`, `STRIPE_PRICE_ID`, `GRUPOS_LIMITE_GRATIS`, `MINIO_ROOT_USER` e `MINIO_ROOT_PASSWORD`. O arquivo `.env.example` do repositório traz só os nomes.

3.3 Fluxo de uma requisição

Exemplo: um líder de grupo cria um novo grupo de estudo.

1. O navegador envia `POST /grupos/novo` ao domínio, por HTTPS, com o cookie de sessão do Flask, que guarda o JWT do usuário.
2. O app chama o auth-service (`/validate-token`) para validar o JWT; o auth-service devolve `user_id`, nome e papel. Sem token válido, o usuário é redirecionado ao login.

3. O app confere o papel: `admin` e `lider` podem criar; `estudante` recebe 403 (e o acesso negado é registrado no log).
4. Se o usuário não é Pro, o app conta no MariaDB os grupos que ele criou no mês; no 4º, responde 403 com a mensagem do limite do plano Free.
5. O app valida os campos (título, disciplina existente, horários, vagas) e faz o `INSERT` em `grupos`, com `lider_id` igual ao id do JWT, nunca ao de um campo do formulário.
6. O app envia o evento `grupo_criado` ao log-service, que o grava no Redis Stream, e responde com redirecionamento para a página do grupo.

3.4 Fluxo de assinatura

O usuário Free clica em *Assinar Pro* (`POST /assinar`). O app cria, se ainda não existir, um Customer no Stripe, guarda o id dele em `assinaturas` e cria uma Checkout Session com `mode=subscription`, levando o id do usuário em `client_reference_id`. O pagamento acontece na tela hospedada pelo Stripe, então nenhum dado de cartão passa pelo servidor nem pelo banco. Ao voltar para o app, o plano só muda quando o webhook chega: a página de sucesso apenas informa.

O endpoint `POST /stripe/webhook` valida a assinatura do cabeçalho `Stripe-Signature` com o segredo do webhook (requisições sem assinatura válida recebem 400 e nada é gravado) e trata quatro eventos: `checkout.session.completed` (guarda customer e subscription e põe o plano em Pro), `customer.subscription.updated` (atualiza status, preço e cancelamento agendado), `customer.subscription.deleted` (volta o usuário ao Free) e `invoice.payment_failed` (marca `past_due` e tira o Pro). Cada `event.id` é gravado em `stripe_eventos` com `INSERT IGNORE`; se o mesmo evento chegar de novo, o app responde 200 e não processa de novo, então o resultado não muda. Se o processamento falha, a marca é desfeita e a resposta é 500, para o Stripe reenviar. O cancelamento é feito pelo portal do cliente (*Gerenciar assinatura*, `POST /assinatura/portal`), que gera `customer.subscription.updated` e `customer.subscription.deleted`. Como toda rota que depende do plano lê o banco a cada requisição, o limite do Free vale na API e não apenas na tela.



Figura 2: Fluxo de assinatura: checkout → webhook → plano no banco → limites.

3.5 Modelo de dados do domínio

As tabelas do negócio são `disciplinas`, `grupos` e `grupo_membros`, criadas pelo script versionado `db/001_dominio.sql`, que o app executa ao iniciar (com a seed das disciplinas); `db/002_assinaturas.sql` cria `stripe_eventos`. As chaves estrangeiras reais são `grupos.disciplina_id` → `disciplinas`, `grupo_membros.grupo_id` → `grupos` (com `ON DELETE CASCADE`) e `reset_tokens.usuario_id` → `usuarios`. A ligação com o usuário (`grupos.lider_id`, `grupo_membros.user_id`, `perfis.user_id` e `assinaturas.user_id`) é lógica, e não uma chave estrangeira, porque a tabela `usuarios` pertence a outro serviço (o auth-service). A tabela `assinaturas` liga o usuário ao plano e aos identificadores do Stripe (`stripe_customer_id`, `stripe_subscription_id`, `stripe_price_id`).

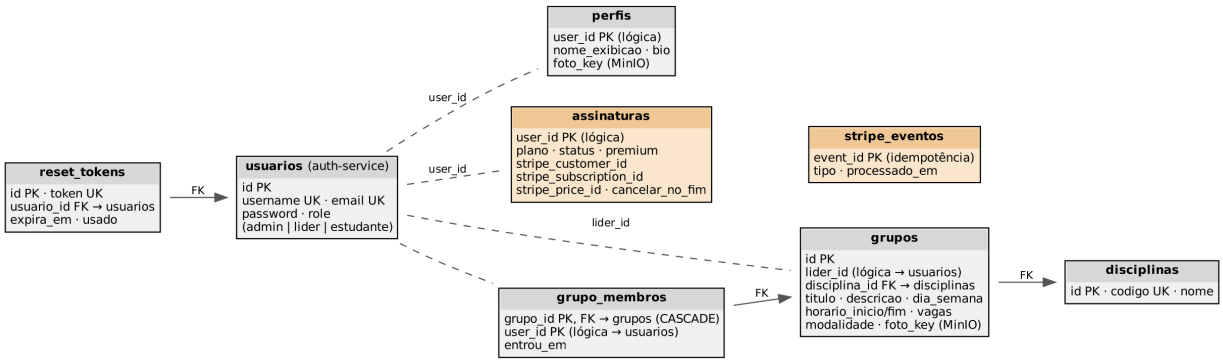


Figura 3: Modelo de dados do domínio (linhas contínuas: chaves estrangeiras; tracejadas: referências lógicas por `user_id`).

3.6 Segurança

Ação	admin	líder	estudante
criar grupo	sim	sim (Free: 3 por mês)	não (403)
editar, excluir e enviar capa do próprio grupo	sim	sim	não (403)
ver o grupo de outro usuário	sim	não (403)	só se for membro
editar ou excluir grupo de outro usuário	sim	não (403)	não (403)
ver `/admin/logs` e `/admin/painel`	sim	não (403)	não (403)
assinar o Pro e abrir o portal do cliente	sim	sim	sim

As permissões valem no backend, e não só na tela: uma chamada direta à API com papel sem permissão recebe 403. O isolamento por usuário é garantido nas consultas e nas funções de acesso: o líder só lista os grupos em que `lider_id` é o seu, o estudante só os de que é membro, e abrir ou alterar o grupo de outra pessoa responde 403 e gera um evento no log. O id do usuário sempre vem do JWT validado pelo auth-service, nunca de campos enviados pelo cliente.

Os segredos do Stripe (`STRIPE_SECRET_KEY` e `STRIPE_WEBHOOK_SECRET`) ficam só nas variáveis de ambiente da stack, e o repositório tem apenas o `.env.example` com os nomes. Há limitações que reconheço: o `docker-compose.yml` ainda traz valores padrão de desenvolvimento para as credenciais do banco, o `JWT_SECRET` e o MinIO, que deveriam ser removidos; e as senhas dos usuários ainda são gravadas em texto puro em `usuarios.password` (o certo seria guardar um hash). Ambas estão registradas como pendências.

3.7 Observabilidade, CI/CD e documentação da API

Não implementei as atividades extras de health checks e métricas, pipeline do GitHub Actions ou Swagger. O que existe: os eventos de negócio e de auditoria no log-service (visíveis ao admin em `/admin/logs`), o painel `/admin/painel` com usuários por plano, assinaturas ativas, MRR de teste e últimos eventos, 61 testes automatizados com pytest e a API REST documentada no README do repositório.

3.8 Camadas IaaS, PaaS e SaaS do projeto

Camada	O que é no meu projeto	Quem gerencia
IaaS	a VM, a rede e o disco onde a stack roda (o servidor compartilhado da disciplina, com IP e domínio públicos)	Fatec / professor
PaaS	Docker e Portainer (plataforma de containers que faz o build e o redeploy a partir do GitHub) e o MariaDB (banco fornecido pronto, do qual uso só o schema)	Fatec (plataforma e banco); eu defino a stack e as tabelas
Auto-hospedado	Redis e MinIO, que subo em containers e opero eu mesmo (volume, imagem, credenciais)	eu

SaaS	o StudyMatch, para o cliente final; e os SaaS que uso: Stripe (pagamentos), GitHub (código e issues de entrega) e Mailtrap (e-mail de teste)	eu (o produto); fornecedores (os serviços)

4 Quadro de entregas

Atividades obrigatórias do 2º bimestre. A data realizada é a do commit (atividades 7 e 8) ou da issue de entrega (atividades 9 a 11). As atividades 8 e 10 foram entregues após a data planejada, o que desconta 0,25 da nota da P2 cada uma, conforme a regra do enunciado.

Nº	Atividade	Data planejada	Data realizada	Situação
7	Plano Premium — cobrança de verdade com Stripe	09/10/2026	08/10/2026 19:35	entregue
8	SaaS, PaaS, IaaS — e a documentação que o projeto nunca teve	16/10/2026	21/10/2026 19:47	entregue com atraso
9	Fork e plano de negócio	23/10/2026	21/10/2026 20:10	entregue
10	Seu negócio no ar — o domínio no lugar dos filmes	06/11/2026	09/11/2026 19:33	entregue com atraso
11	SaaS completo — planos, limites e assinatura com Stripe	13/11/2026	11/11/2026 19:51	entregue

5 Atividades realizadas

5.1 Atividade 7 — Plano Premium — cobrança de verdade com Stripe

Descrição	Plano pago em modo de teste: Checkout Session do Stripe e webhook marcando o usuário como premium.
Data planejada	09/10/2026
Data realizada	08/10/2026 19:35
Situação	entregue
Evidência	GitHub — commit + print do checkout concluído e do usuário premium
Link	https://github.com/NicollasZoratto/Smart-STUDY-Match/commit/94dd5c7e09c3723628669314fa9a4be35d176c96

O que foi feito. Criei o plano pago em modo de teste do Stripe (R$ 9,90 por mês) e a rota que cria uma Checkout Session para o usuário logado e o redireciona ao pagamento. O webhook recebe a confirmação, valida a assinatura (`Stripe-Signature`) e só então marca o usuário como premium na tabela `assinaturas`, que guarda apenas o status e os ids do Stripe. Decidi que o Premium nunca é concedido pelo redirecionamento de volta do checkout, e sim somente pelo webhook assinado. O benefício verificável era o limite de favoritos no plano grátis, aplicado no backend, como a diferença entre admin e usuário da atividade 4.

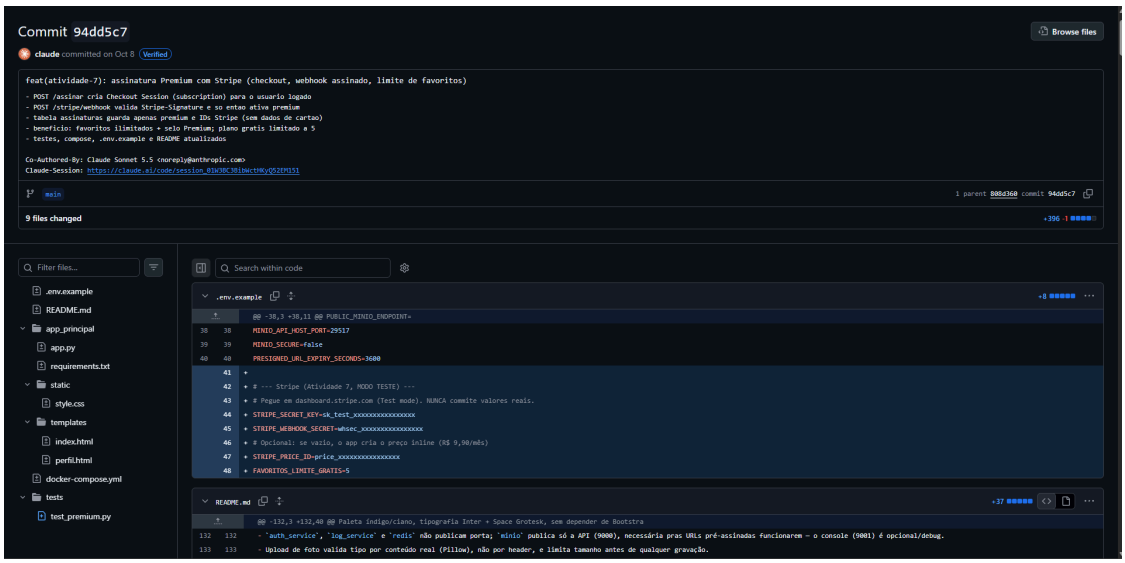


Figura 4: Atividade 7 — evidência da entrega (link, data e hora visíveis).

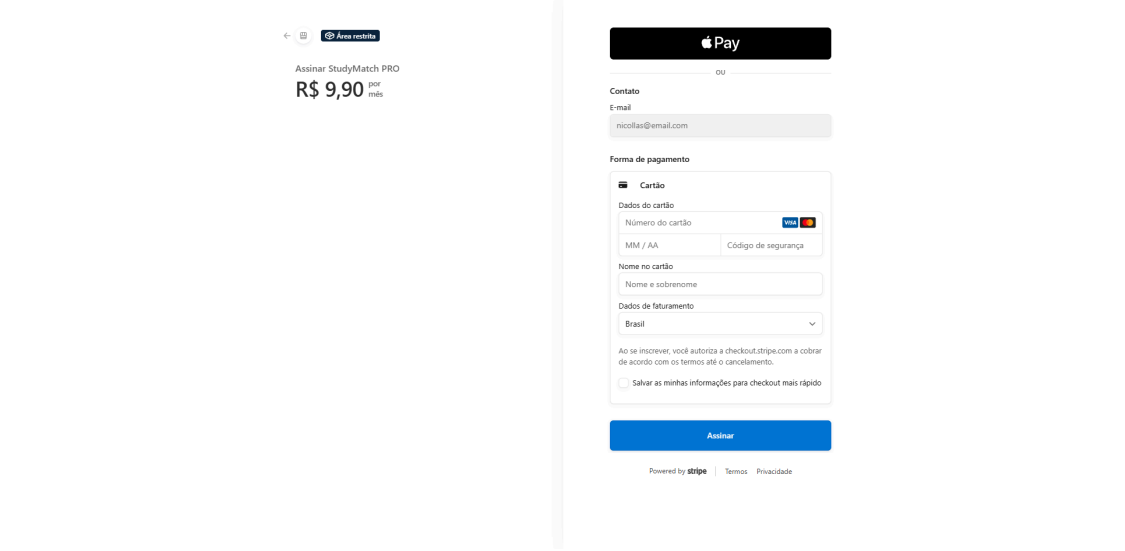


Figura 5: Atividade 7 — resultado (o sistema funcionando).

Dificuldades e como foram resolvidas. O Stripe normalmente exige uma URL HTTPS pública para o webhook, e o servidor era acessado por IP e porta; usei o endereço HTTPS do domínio da disciplina. Também foi preciso receber o corpo cru da requisição para validar a assinatura, e testei isso com um payload assinado de verdade (HMAC) nos testes automatizados.

5.2 Atividade 8 — SaaS, PaaS, IaaS — e a documentação que o projeto nunca teve

Descrição	Camadas de nuvem do próprio projeto e documentação reversa: especificação, arquitetura, regras de negócio e modelo de dados.
Data planejada	16/10/2026
Data realizada	21/10/2026 19:47
Situação	entregue com atraso
Evidência	GitHub — commit + documentação linkada no README
Link	https://github.com/NicollasZoratto/Smart-STUDY-Match/commit/cdb7a74e446c980b4e6171ed7e4ad77c4f0e2c98

O que foi feito. Escrevi no README a classificação de cada peça do sistema em IaaS, PaaS, CaaS e SaaS, com uma frase de justificativa, e a documentação reversa feita lendo o código: especificação do ponto de vista de quem usa, diagrama de arquitetura, regras de negócio no formato "se X, então Y" e modelo de dados. A leitura do código também revelou problemas, que ficaram registrados em "Limitações conhecidas": senhas em texto puro, a tabela de usuários apagada a cada reinício do auth-service e o código 2FA sem prazo de validade.

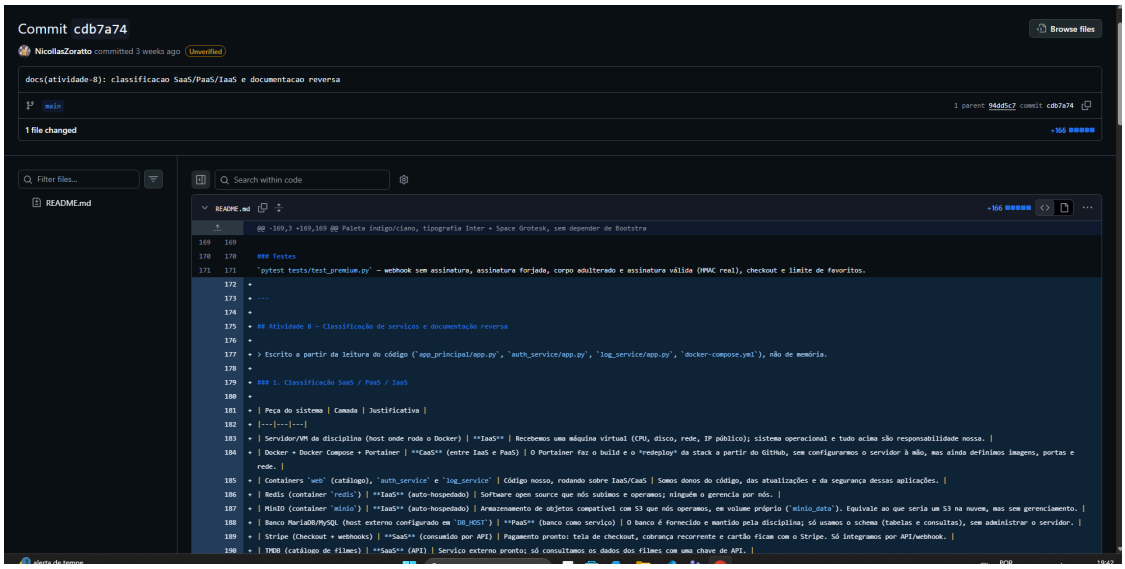


Figura 6: Atividade 8 — evidência da entrega (link, data e hora visíveis).

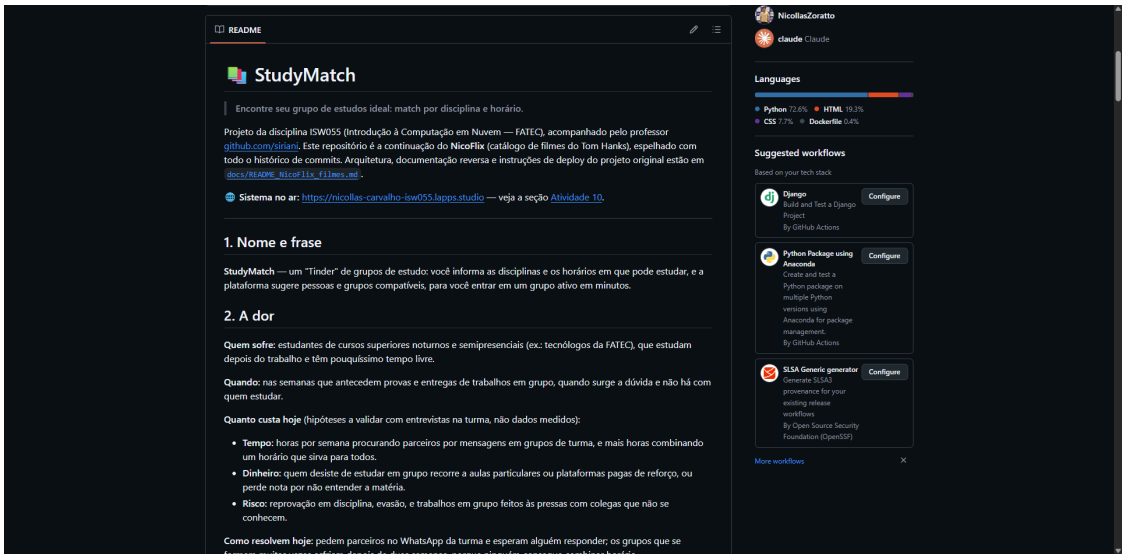


Figura 7: Atividade 8 — resultado (a documentação publicada no README).

Dificuldades e como foram resolvidas. Extrair as regras do código, e não da memória, exigiu ler as rotas e os validadores de cada serviço. **Entregue com atraso** (planejada em 16/10, realizada em 21/10): a documentação foi escrita só no dia 21/10, depois de priorizar a estabilização da stack no Portainer.

5.3 Atividade 9 — Fork e plano de negócio

Descrição	Repositório novo com o histórico do anterior, README do produto (dor, persona, Lean Canvas, modelo de receita, mapa de reaproveitamento) e backlog em issues.
Data planejada	23/10/2026
Data realizada	21/10/2026 20:10
Situação	entregue
Evidência	GitHub — issue de entrega (\@siriani) + repositório novo + README do produto + prints do histórico e das issues
Link	https://github.com/NicollasZoratto/Smart-STUDY-Match/issues/9

O que foi feito. Criei o repositório novo e público do produto com todo o histórico do repositório anterior (a cópia espelhada pedida no enunciado, já que o botão *Fork* do GitHub não copia um repositório para a mesma conta) e troquei o README: nome e frase do StudyMatch, a dor, a persona (Camila, 21 anos, aluna noturna de ADS), cinco concorrentes e alternativas, a proposta de valor, o modelo de receita com Stripe (Free e Pro), as métricas de sucesso, o Lean Canvas com os 9 blocos e o mapa de reaproveitamento de cada peça do projeto atual. Abri 8 issues de backlog e a issue de entrega, mencionando o professor, e deixei no README antigo uma linha apontando para o repositório novo.

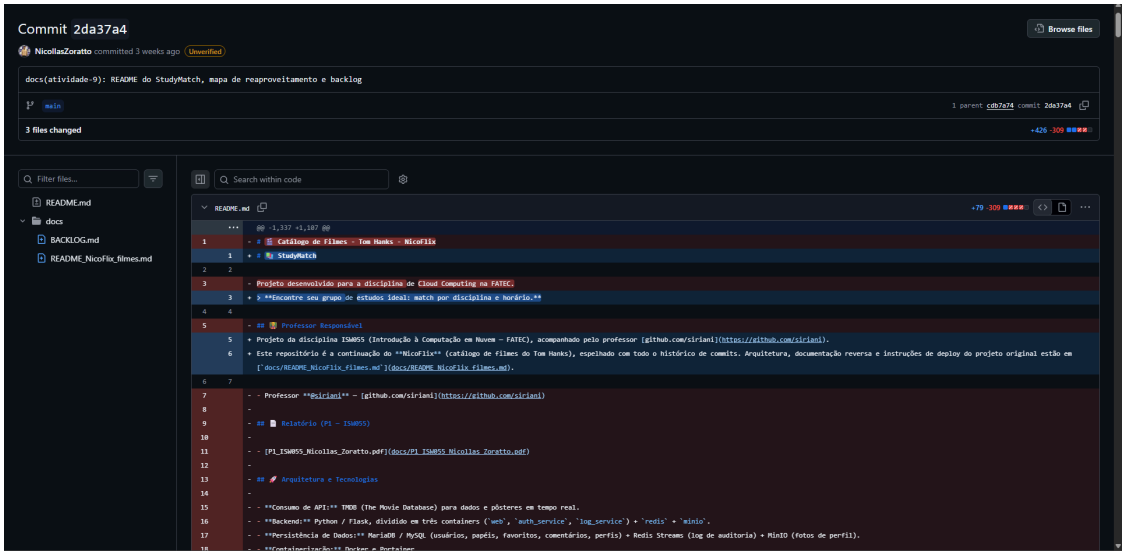


Figura 8: Atividade 9 — evidência da entrega (link, data e hora visíveis).

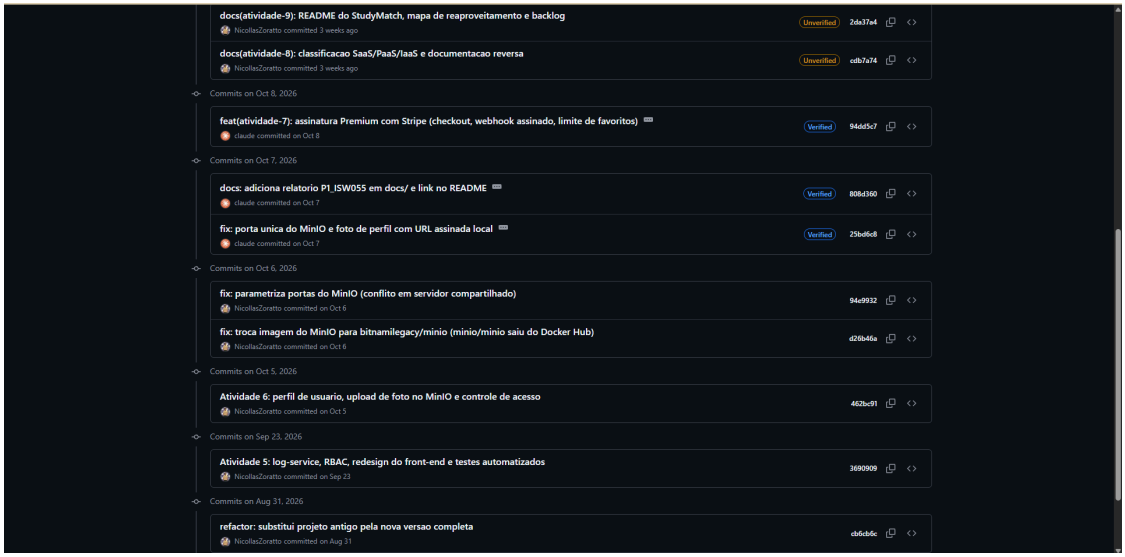


Figura 9: Atividade 9 — resultado (histórico de commits do repositório novo e lista de issues).

Dificuldades e como foram resolvidas. Escolher um negócio plausível e quantificar a dor sem inventar dados: marquei os números de custo como hipóteses a validar com entrevistas na turma. O nome do repositório (`Smart-STUDY-Match`) ficou diferente do nome do produto no README (StudyMatch), o que não afeta o sistema.

5.4 Atividade 10 — Seu negócio no ar — o domínio no lugar dos filmes

Descrição	Stack apontando para o repositório novo, tabelas e API do domínio, papéis do negócio, isolamento por usuário, eventos no log, upload e landing page.
Data planejada	06/11/2026
Data realizada	09/11/2026 19:33
Situação	entregue com atraso
Evidência	GitHub — issue de entrega (\@siriani) + commit + URL no ar + prints da landing, do 403 e dos eventos no log
Link	https://github.com/NicollasZoratto/Smart-STUDY-Match/issues/10

O que foi feito. Troquei o repositório da stack do Portainer para o novo, sem criar stack nem trocar de subdomínio, e substituí o catálogo de filmes pelo recurso principal do negócio, o grupo de estudo. Criei as tabelas `disciplinas`, `grupos` e `grupo_membros` com chaves estrangeiras e o script de migração e seed `db/001_dominio.sql`, a API REST com CRUD completo em `/api/grupos`, as telas de listar, criar, detalhar e editar, os três papéis no auth-service (`admin`, `lider` e `estudante`, escolhidos no cadastro), o isolamento por usuário (403 ao abrir o grupo de outra pessoa), eventos de negócio no log e o upload da capa do grupo no MinIO, ligado ao registro certo. A landing page pública ficou na raiz do domínio, e removi o código, as telas, as rotas e a variável do TMDB.

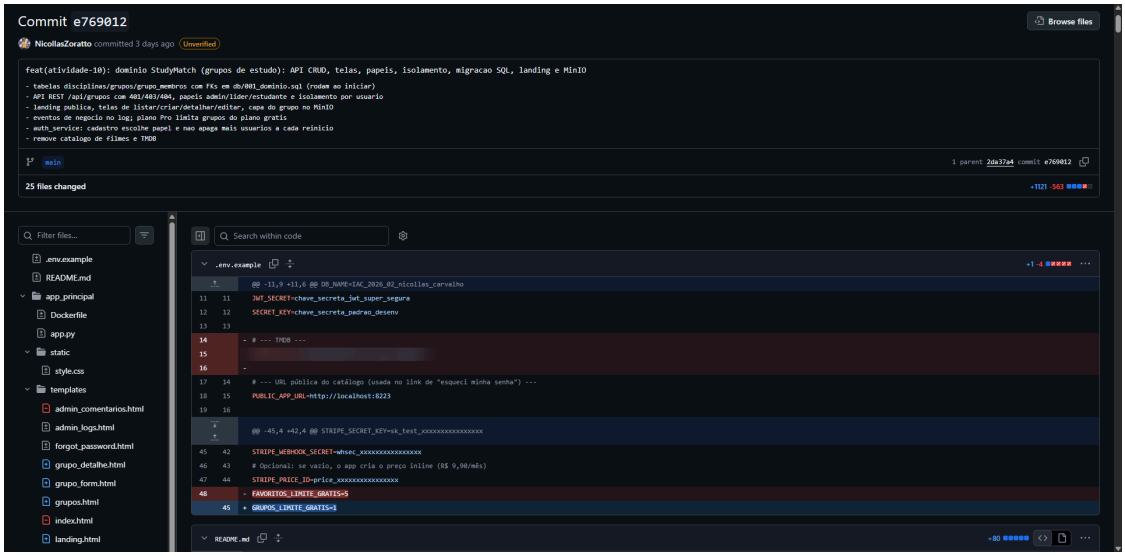


Figura 10: Atividade 10 — evidência da entrega (link, data e hora visíveis).

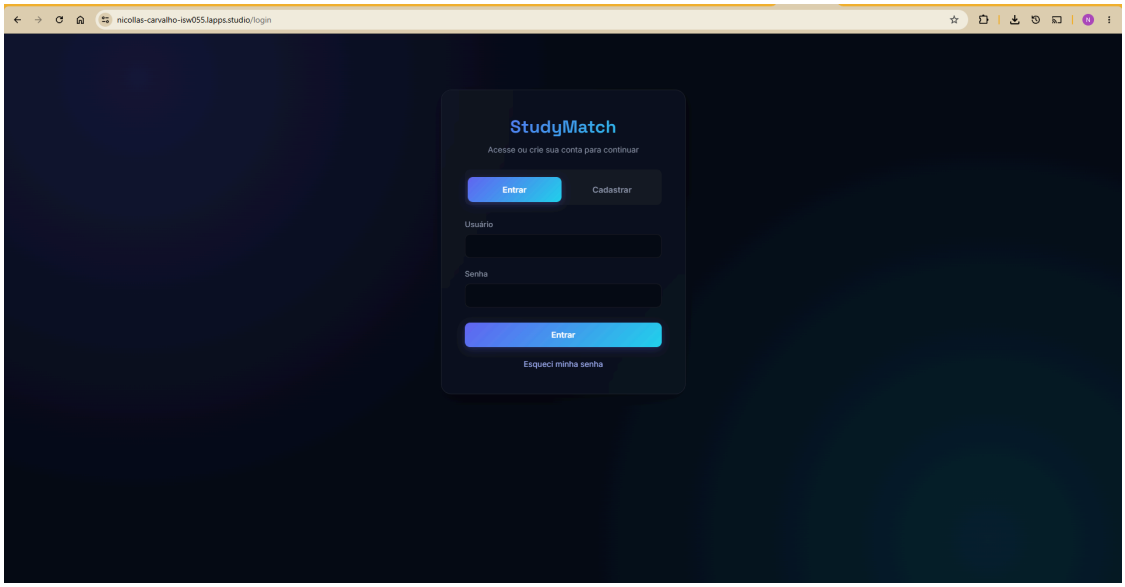


Figura 11: Atividade 10 — tela pública de acesso (login e cadastro com escolha de papel) no ar.

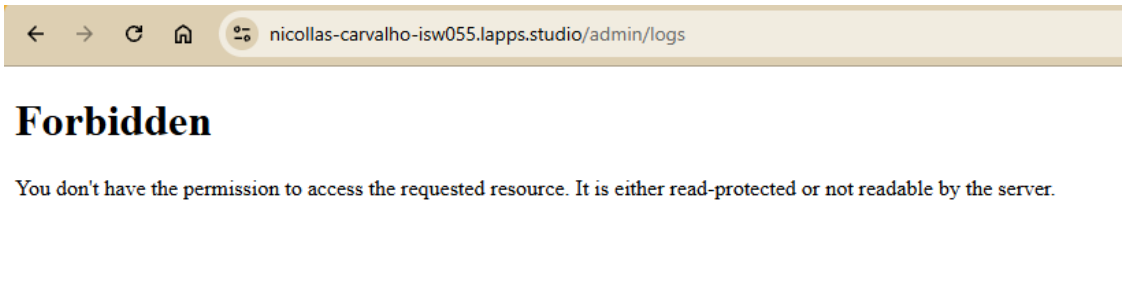


Figura 12: Atividade 10 — resposta 403 (Forbidden) quando um estudante abre a tela restrita /admin/logs.

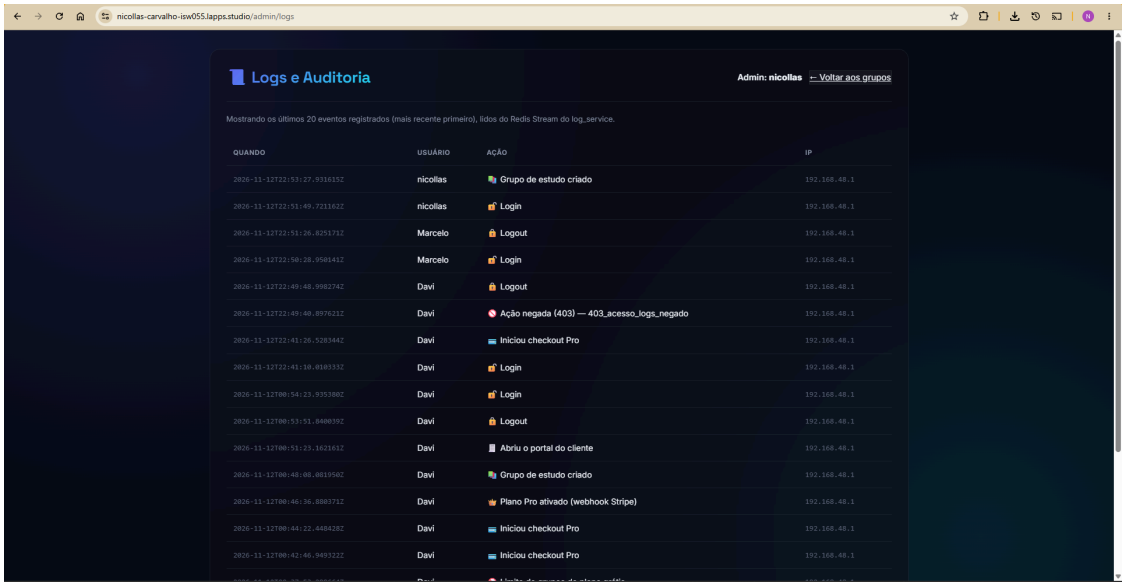


Figura 13: Atividade 10 — eventos de negócio no log (grupo criado, login, checkout, ação negada 403), vistos pelo admin.

Dificuldades e como foram resolvidas. Os identificadores dos usuários precisavam ser estáveis, mas o auth-service apagava a tabela de usuários a cada reinício; passei a criá-la só se não existir e migrei o papel antigo `usuario` para `estudante`. Como `usuarios` pertence a outro serviço, `grupos.lider_id` é uma referência lógica e não uma chave estrangeira. **Entregue com atraso** (planejada em 06/11, realizada em 09/11): a migração do domínio, a troca de papéis e o redeploy foram concluídos apenas na noite de 09/11.

5.5 Atividade 11 — SaaS completo — planos, limites e assinatura com Stripe

Descrição	Planos recorrentes, checkout de assinatura, webhooks atualizando o plano, portal do cliente, limites por plano na API e painel admin.
Data planejada	13/11/2026
Data realizada	11/11/2026 19:51
Situação	entregue
Evidência	GitHub — issue de entrega (\@siriani) + commit + prints da assinatura com o cartão 4242, do Stripe em modo de teste, do webhook e do limite bloqueando
Link	https://github.com/NicollasZoratto/Smart-STUDY-Match/issues/11

O que foi feito. Transformei o pagamento único em assinatura recorrente: plano Free (R$ 0,00, líder cria até 3 grupos por mês) e Pro (R$ 9,90 por mês, sem limite), com o produto e o preço criados no Stripe em modo de teste. O checkout usa `mode=subscription` e fica ligado a um Customer do Stripe guardado no banco; os quatro webhooks mantêm plano, status e ids na tabela `assinaturas`, com validação de assinatura e sem efeito duplicado (tabela `stripe_eventos`). O botão *Gerenciar assinatura* abre o portal do cliente, onde ele troca de plano e cancela. O limite do Free é aplicado na API, que recusa o 4º grupo do mês com 403 e uma mensagem dizendo qual é o limite e como liberar. O painel `/admin/painel`, só para admin, mostra usuários por plano, assinaturas ativas, MRR de teste e últimos eventos.

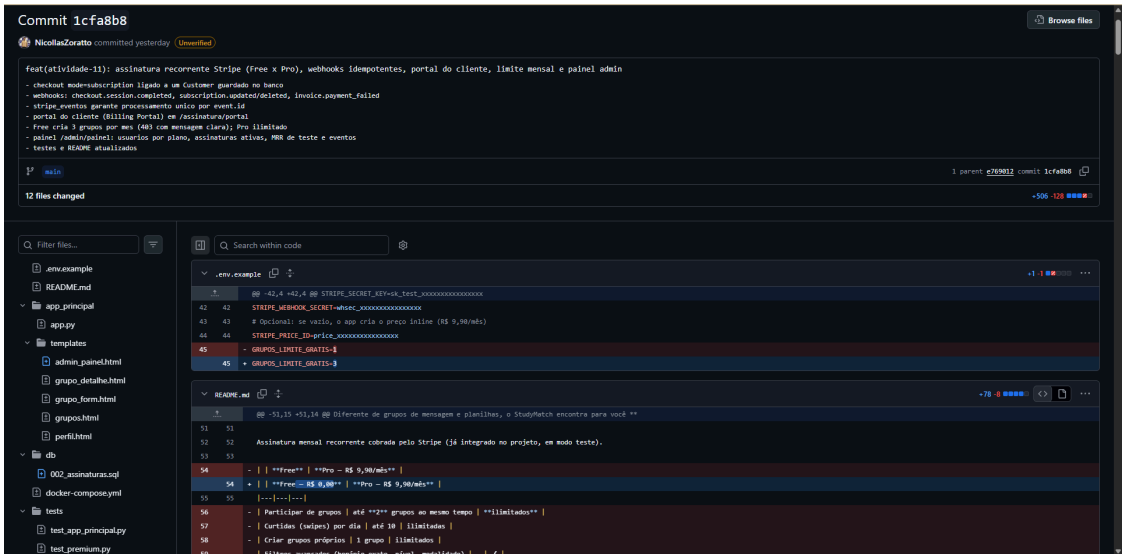


Figura 14: Atividade 11 — evidência da entrega (link, data e hora visíveis).

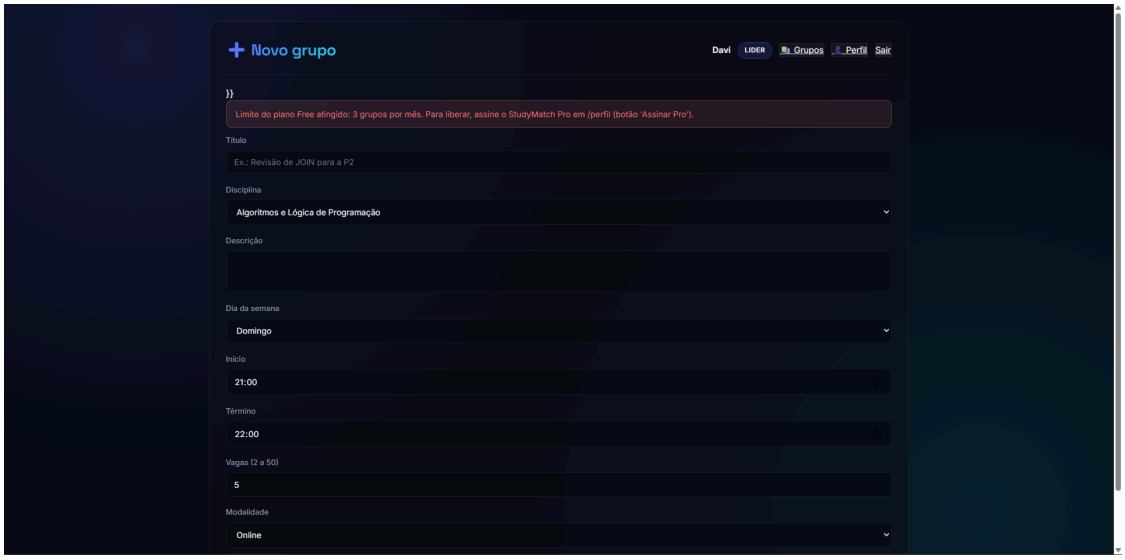


Figura 15: Atividade 11 — limite do plano Free bloqueando o 4º grupo.

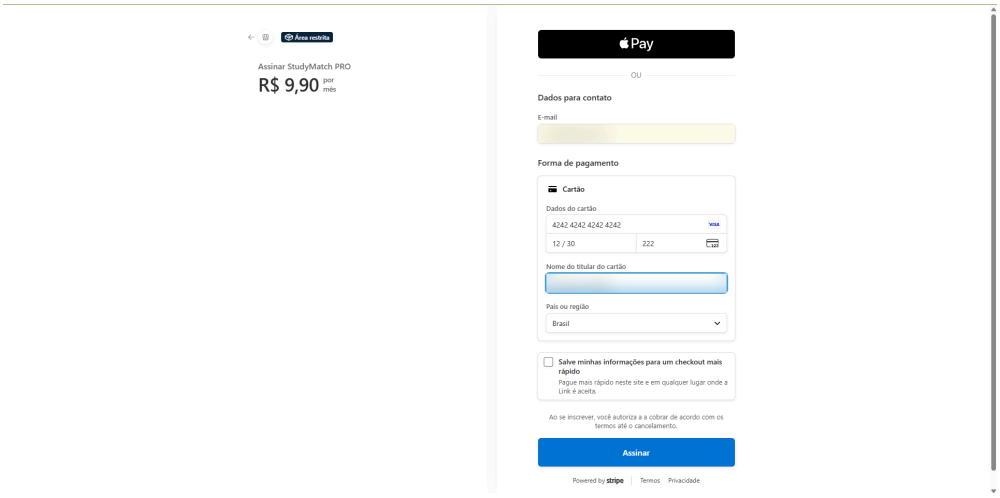


Figura 16: Atividade 11 — checkout de assinatura com o cartão de teste 4242 (e-mail e nome borrados).

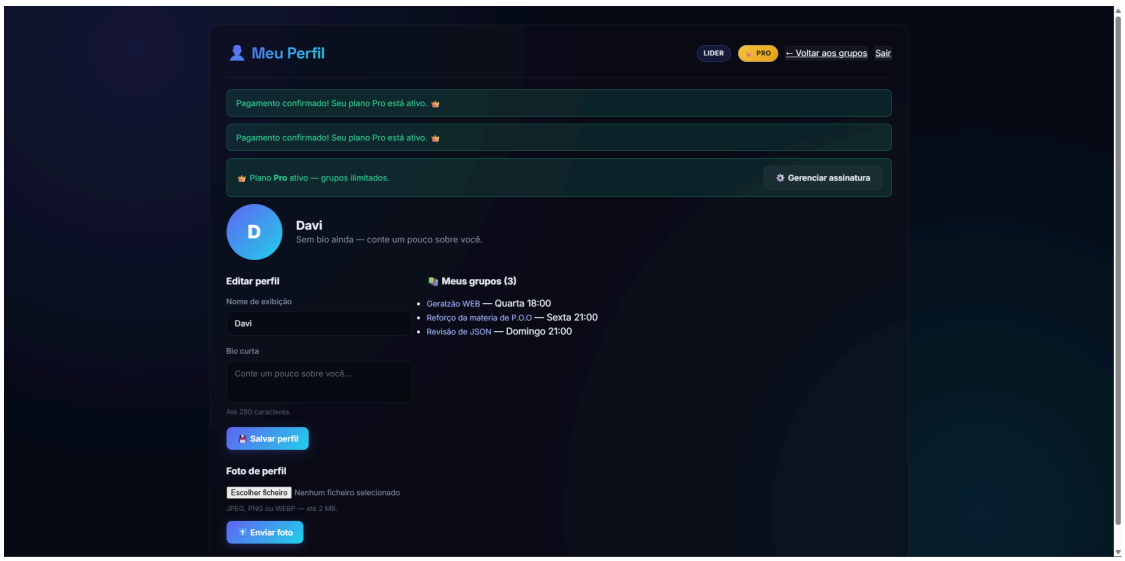


Figura 17: Atividade 11 — perfil já no plano Pro depois do webhook.

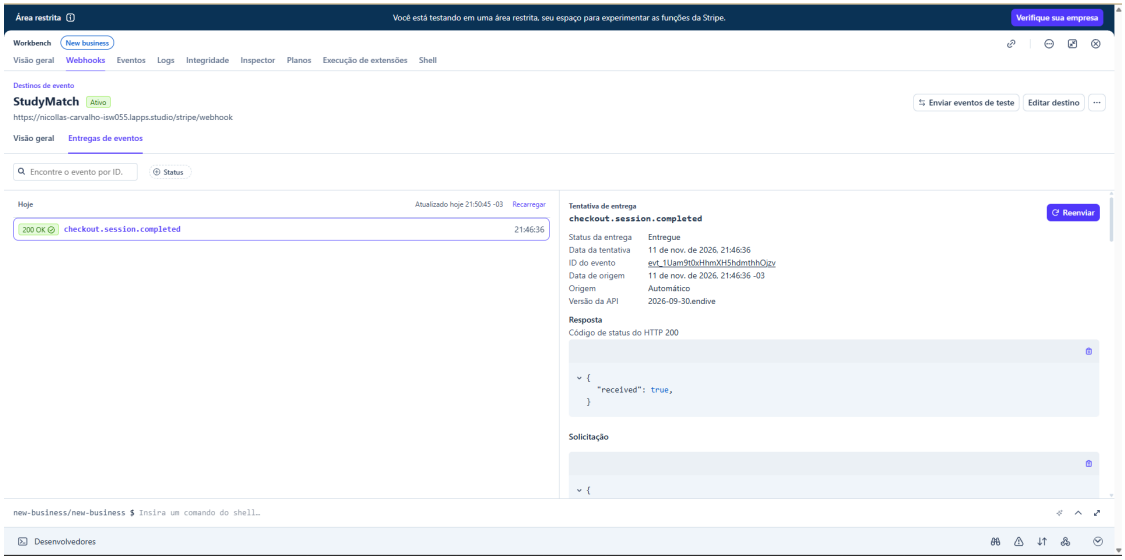


Figura 18: Atividade 11 — webhook `checkout.session.completed` entregue, com resposta 200.

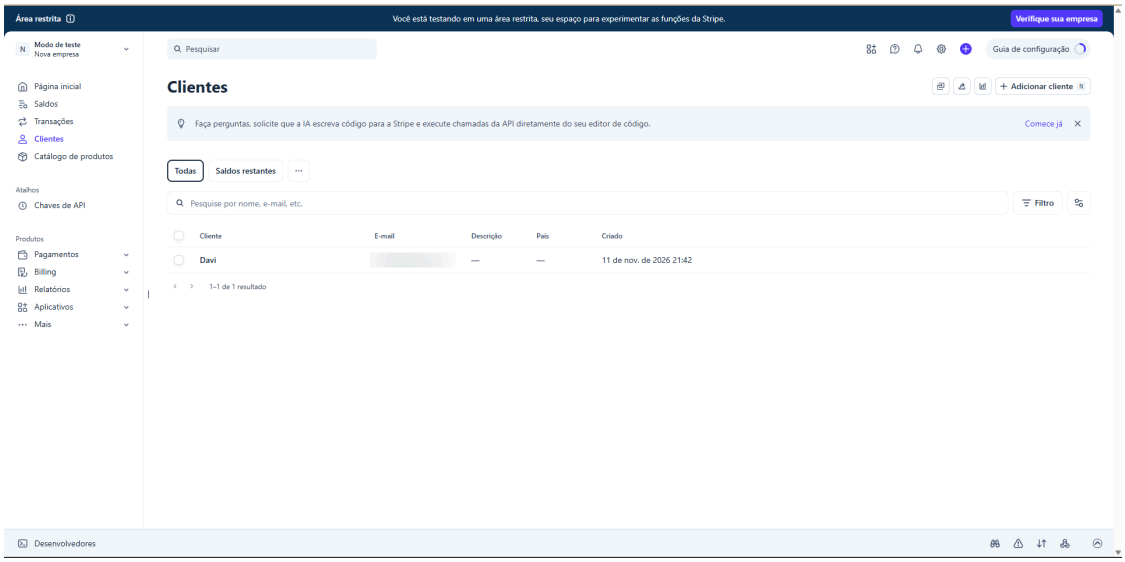


Figura 19: Atividade 11 — Stripe em modo de teste: cliente criado (e-mail borrado).

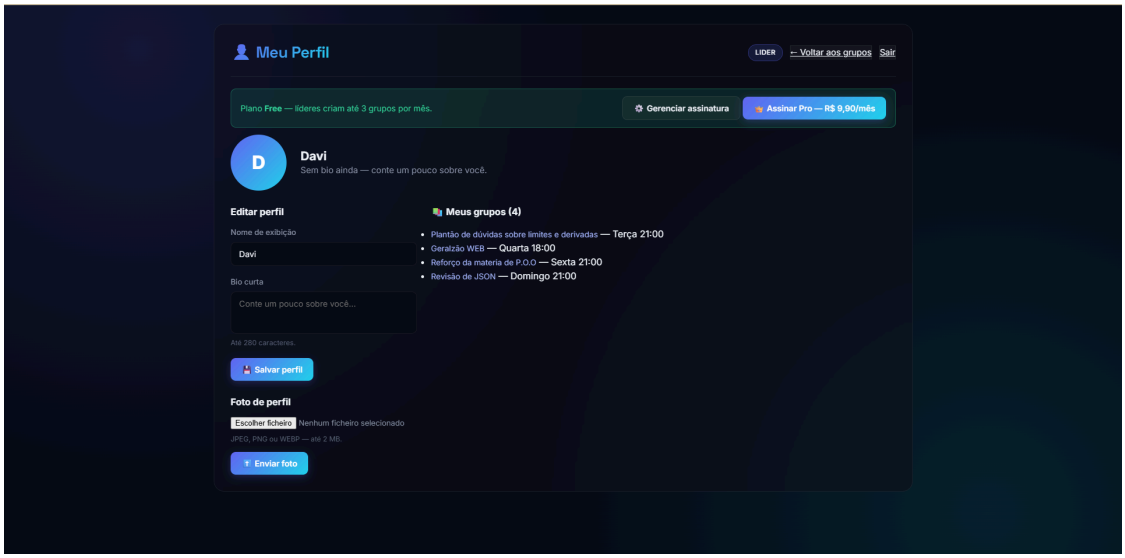


Figura 20: Atividade 11 — conta de volta ao Free depois do cancelamento.

Dificuldades e como foram resolvidas. O redeploy no Portainer falhou com `no space left on device`: o disco do servidor compartilhado estava cheio de imagens antigas. Resolvi removendo as imagens e containers meus que não estavam em uso e repetindo o *Pull and redeploy*. Também foi preciso configurar o portal do cliente no Stripe (cancelamento imediato, para testar na hora) e garantir a idempotência dos webhooks. O print do painel admin não foi incluído nesta versão do relatório.

6 Considerações finais

Neste bimestre aprendi que uma assinatura é um estado, e não um pagamento: o dinheiro entra pelo Stripe, mas quem mantém o plano certo no meu banco são os webhooks, que precisam ser validados, idempotentes e tolerantes à ordem de chegada. Aprendi também a separar o que é do app do que é de outro serviço (por exemplo, a tabela de usuários), a aplicar permissões e isolamento no backend e a documentar um sistema lendo o código real. Entreguei duas atividades com atraso (8 e 10), por ter deixado a documentação e a migração do domínio para o fim.

Para o StudyMatch ser um produto de verdade, ainda faltam: o *match* em si (o estudante descobrir e pedir entrada em grupos compatíveis, hoje ele só vê os grupos em que já é membro); guardar as senhas com hash; remover os valores padrão de credenciais do `docker-compose.yml`; expirar o código 2FA e deixar de mostrá-lo na tela; e as extras que não fiz, como health checks, CI e Swagger. Se eu fosse recomeçar, começaria pelos testes e pela separação de segredos, para não ter que corrigir depois.

7 Declaração de autoria

Declaro que este relatório foi elaborado por mim, individualmente, e que as evidências apresentadas correspondem a entregas de minha autoria, verificáveis nos links informados.

Nicollas Zoratto de Carvalho

Pompeia, 12/11/2026